Docker to Kubernetes Deployment Workflow

Part 1 — Docker Hub nunchi nginx image
Docker Hub lo already official nginx image undi.
Mana local machine ki techukovali:
docker pull nginx
Idi em chestundi?
Docker Hub
→ nginx image
→ **nee local machine**
Ippudu local machine lo:
nginx:latest
ane image undi.
Check cheyyadaniki:
docker images
Ikkada nginx kanipistundi.
Important
Ikkada **build cheyyalsina avasaram ledu**.
Endukante Docker Hub lo already build chesina nginx image undi.
So:
pull = registry nunchi image techukovadam
build = Dockerfile nunchi kotha image tayaru cheyyadam
Rendu different.

Part 2 — nginx image nunchi container create cheyyadam
Ippudu image local lo undi.
Image ni actual ga run cheyyali ante **container create cheyyali**.
Command:
docker run --name container1 -p 8080:80 nginx
Ikkada:

- docker → Docker
- run → image nunchi container create + start
- --name container1 → container ki container1 ane name
- -p 8080:80 → laptop 8080 → container 80
- nginx → **image**

So Docker internally:
nginx image
↓
container create
↓
container1
↓
container start
↓
nginx process running.
-t **enti?**
Nee example lo:
docker run -t ...
-t terminal allocate cheyyadaniki. Nginx web server ki normally idi necessary kaadu.
Interactive terminal kavali ante -it use chestam.
Nginx ki simple ga:
docker run --name container1 -p 8080:80 nginx
chaalu.

Part 3 — Container lopala em undi?
Ippudu:
container1
running.
Container lopala nginx application undi.
Rough ga:
container1
→ filesystem
→ nginx binaries
→ nginx configuration
→ nginx HTML files
→ nginx process
Docker image nunchi container create chesinappudu image lo unna filesystem layers ni container use chestundi.

Part 4 — Ippudu Dockerfile enduku?
Ikkada nee question:
"Tarvatha Dockerfile lo em rayali?"
Nginx image ni simply run cheyyadaniki Dockerfile avasaram ledu.
Dockerfile rayadam eppudu?
Suppose nuvvu nginx ni customize cheyyali.
Example:
Nee own HTML page kavali:
Hello Hyderabad
Appudu Dockerfile create chestav.
Dockerfile lo roughly:
FROM nginx
Meaning:
Existing nginx image ni base ga use cheyyi.
Then nee custom files nginx image loki copy cheyyali.
For example:
COPY index.html /usr/share/nginx/html/index.html
Meaning:
Mana local index.html ni nginx container lo website files unde location ki copy cheyyi.
Then build:
docker build -t mynginx:1.0 .
Ippudu:
Dockerfile
↓
Docker build
↓
mynginx:1.0
ane **new Docker image**.
So ippudu manaki:
nginx = original image
mynginx:1.0 = nee custom image

Part 5 — Custom image ni local ga run cheyyadam
Ippudu custom image create ayyindi:
mynginx:1.0
Daani run cheyyali:
docker run --name mycontainer -p 8080:80 mynginx:1.0
Flow:
Dockerfile
↓
mynginx:1.0 image
↓
mycontainer
↓
nginx running.
Browser:
localhost:8080

Part 6 — Ippudu Kubernetes ki move
Ikkada oka **very important thing**.
Nee laptop lo:
mynginx:1.0
image undi.
Kani EKS Node nee laptop lo unna image ni access cheyyadu.
So image ni **registry** ki push cheyyali.
Registry example:
Amazon ECR
Flow:
Local Docker
mynginx:1.0
↓
tag
↓
ECR image name
↓
push
↓
Amazon ECR
Ippudu image AWS registry lo undi.

ECR enti?

Amazon ECR = AWS lo Docker images store cheskune place.
Nee local lo:
mynginx:1.0
undi.
EKS Node nee laptop image ni direct ga access cheyyaledu. So image ni ECR lo pettali.
Flow
1. ECR lo repository create cheyyi
Example:
mynginx
2. Local image ni ECR address tho tag cheyyi
mynginx:1.0
→ AWS-ECR-ADDRESS/mynginx:1.0
3. AWS ECR ki Docker login cheyyi
4. Push cheyyi
docker push AWS-ECR-ADDRESS/mynginx:1.0
Appudu:
Local image → ECR
Aipoyindi.
Tarvatha EKS
Kubernetes YAML lo:
image: AWS-ECR-ADDRESS/mynginx:1.0
ani pedtham.
Appudu:
EKS Node → ECR nunchi image pull → container create → Pod lo run.
Anthe.

Part 7 — EKS Cluster create
AWS lo EKS create chestam.
Conceptually:
AWS
↓
EKS Cluster
Cluster lo two major sides:
Control Plane
and
Worker Nodes
EKS lo AWS control plane ni manage chestundi.

Part 8 — Nodes create
Cluster create ayyaka application run cheyyadaniki compute kavali.
Anduke **Node Group** create chestam.
Node Group example:
my-node-group
It creates/maintains worker nodes.
Example:
EKS Cluster
→ Node 1
→ Node 2
→ Node 3
Each Node is a machine where Pods can run.
Node lo important:
kubelet
and
container runtime
Modern EKS nodes lo commonly containerd.

Part 9 — Kubernetes lo Pod create cheyyadam
Ippudu cluster + nodes ready.
Nuvvu Kubernetes ki chepthav:
"Na application ni run cheyyali."
Daaniki Kubernetes object create chestam.
Beginner ga direct **Pod YAML** tho chuddam.
YAML lo:

- Pod name
- container name
- image
- container port

define chestam.

Main line:
image: mynginx:1.0
Idi:
"Ee Pod lo container create chestappudu mynginx:1.0 image use cheyyi."
Then:
kubectl apply -f pod.yaml

Part 10 — kubectl apply **tarvatha exact ga em jarugutundi?**
Ikkada interesting part.
Nuvvu:
kubectl apply -f pod.yaml
chestav.
Request:
kubectl
↓
EKS API Server
ki velthundi.
API Server:
"Okay, ee Pod create cheyyali."
Pod object Kubernetes lo create avtundi.
Kani Pod inka actual Node lo run avvaledu.

Part 11 — Scheduler
Kubernetes Scheduler chustundi:
"Ee Pod ni ye Node lo run cheyyali?"
Suppose:
Node 2
select ayyindi.
Then:
Pod → Node 2
assignment avtundi.

Part 12 — Node lo kubelet
Node 2 lo kubelet already running.
Kubelet ki telustundi:
"Na Node ki ee Pod assign ayyindi."
Pod specification chustundi.
Andulo:
image: mynginx:1.0
undi.
Kubelet container runtime ni contact chestundi.

Part 13 — Container runtime
Node 2 lo:
containerd
untundi.
Kubelet:
"Ee Pod kosam container create cheyyi."
Containerd:
"Okay, image ekkada undi?"
mynginx:1.0
image Node lo already lekapothe:
Amazon ECR
ki velthundi.

Part 14 — ECR nunchi image pull
Flow:
Node 2
↓
containerd
↓
"My image local lo ledu"
↓
Amazon ECR
↓
mynginx:1.0
↓
Node 2 local image store.

Part 15 — Image nunchi container
Ippudu image Node lo undi.

Containerd aa image ni use chesi:
container create
chestundi.
So:
mynginx:1.0 image
↓
mynginx container
And aa container:
nginx-pod
lopala run avtundi.
Final:
Node 2
→ **nginx-pod**
→ **mynginx container**
→ **mynginx:1.0 image**

Part 16 — Mari ReplicaSet enti?
Ippudu direct Pod create cheyyadam production lo generally best practice kaadu.
Suppose nuvvu cheppav:
"Naaku nginx 3 copies kavali."
Pod manually 3 create cheyyadam bad idea.
Instead:
Deployment create chestam.
Deployment lo:
replicas: 3
ani specify chestam.

Part 17 — Deployment → ReplicaSet → Pods
Ikkada hierarchy very important:
Deployment
↓
ReplicaSet
↓
Pods
↓
Containers
↓
Images
Example:
nginx Deployment
replicas: 3
↓
ReplicaSet
↓
Pod 1
Pod 2
Pod 3
↓
Each Pod:
nginx container
↓
Each container:
mynginx image

Part 18 — ReplicaSet actual job enti?
ReplicaSet ki oka simple responsibility:
"Desired number of Pods always running lo undali."
Suppose:
replicas: 3
Current:
3 Pods
Everything okay.
Suppose Pod 2 crash ayyindi.
Now:
2 Pods
ReplicaSet notices:
"Naku 3 kavali, ippudu 2 unnayi."
So new Pod create chestundi.
Now:
Pod 1
Pod 3

Pod 4
Again 3.
That's ReplicaSet purpose.

Part 19 — Deployment enduku?
Deployment ReplicaSet ni directly manage cheyyadaniki manaki convenient abstraction.
Suppose:
Current:
mynginx:1.0
3 Pods.
Nuvvu new version release chesav:
mynginx:2.0
Deployment update chesthe Kubernetes controlled rollout chestundi.
Old Pods slowly replace avthai.
So:
Deployment = application rollout/update management
ReplicaSet = required Pod count maintain
Pod = container run ayye unit
Container = image nunchi actual running process/environment

Part 20 — Deployment create chesaka exact flow
Nuvvu:
kubectl apply -f deployment.yaml
chestav.
Then:
API Server
↓
Deployment object
↓
Deployment controller
↓
ReplicaSet create
↓
ReplicaSet says:
"3 Pods kavali."
↓
Pods create
↓
Scheduler each Pod ki Node choose chestundi
↓
Kubelet on selected Node
↓
containerd
↓
ECR nunchi image pull
↓
mynginx:1.0
↓
container create
↓
container start

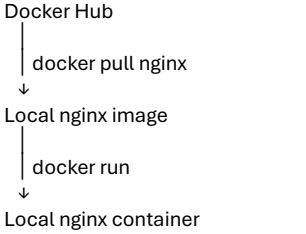
Part 21 — Suppose 3 Nodes unnayi
Example:
Cluster
├— Node 1
├— Node 2
└— Node 3
Deployment:
replicas = 3
Kubernetes scheduler decide cheyyachu:
Pod 1 → Node 1
Pod 2 → Node 2
Pod 3 → Node 3
Then:
Node 1
→ Pod 1
→ nginx container
Node 2
→ Pod 2
→ nginx container

Node 3
→ Pod 3
→ nginx container
Same image:
mynginx:1.0

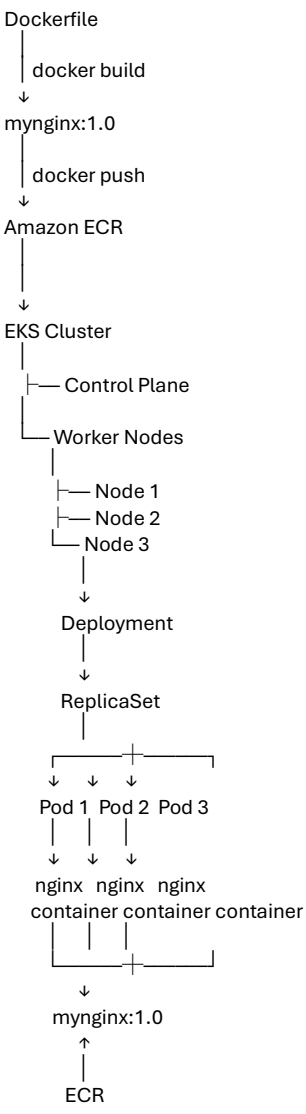
Part 22 — Ippudu user traffic ela vastundi?
Pod ki direct ga browser nunchi traffic pampadam usually cheyyam.
Kubernetes lo **Service** create chestam.
Service:
"Ee nginx Pods ki traffic pampu."
So:
Browser → **Load Balancer / Service** → Pod 1 / Pod 2 / Pod 3 → nginx containers

---

Part 23 — Final complete architecture



Own/custom image case:



Part 24 — Service
Step 1 — Service file create cheyyali
Em cheyyali?
nginx-service.yaml ane file create cheyyali.
Enduku?
Kubernetes ki **"naa nginx Pods ki network entry point create cheyyi"** ani cheppadaniki.
File lo:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer
```

Important: Nee Pods ki app: nginx label undali. Lekapothe Service ki ee Pods yevo teliyavu.

Step 2 — Service create cheyyali
Em cheyyali?
kubectl apply -f nginx-service.yaml
Enduku?
File lo rasina Service configuration ni Kubernetes cluster lo actual ga create cheyyadaniki.
Result:
Kubernetes lo:
nginx-service
ane Service create avtundi.

Step 3 — Service create ayyindo check cheyyali
Command:
kubectl get svc
Enduku?
Service successfully create ayyinda, TYPE, PORT, EXTERNAL-IP enti ani chudataniki.

Step 4 — AWS Load Balancer create avvali
Manam YAML lo:
type: LoadBalancer
pettam.
Enduku?
Internet nunchi nginx ni access cheyyali.
Kubernetes/AWS integration AWS lo external Load Balancer provision chestundi.
Manam separate aws **command tho Load Balancer create cheyyam** in this basic flow.
Check:
kubectl get svc nginx-service
EXTERNAL-IP / hostname kosam wait cheyyali.

Step 5 — Service ye Pods ni use chestundo check cheyyali
Command:
kubectl get endpoints nginx-service
Enduku?
Service app=nginx selector tho **ye Pods ni backend ga select chesindo** chudataniki.
Example ga Pod IPs kanipistayi:
10.x.x.x:80
10.x.x.x:80
10.x.x.x:80
Ante Service ki nginx Pods dorikayi.

Step 6 — Browser nunchi test cheyyali
kubectl get svc nginx-service
lo vachina **external hostname** teesukoni browser lo open cheyyi.
Enduku?
Actual complete path test cheyyadaniki:
Browser → AWS Load Balancer → Service → nginx Pod → nginx container
Nginx welcome page vaste **Service successfully work avtundi.**

Step 7 — Pod delete chesi Service behavior test cheyyali
Suppose 3 Pods unnayi.
Pod name telusukovali:
kubectl get pods

Oka Pod delete:
kubectl delete pod <pod-name>
Enduku?
Deployment/ReplicaSet replacement Pod create chestundo, Service new Pod ni automatically pick chestundo chudataniki.
Check:
kubectl get pods
Malli 3 Pods untayi.
Service:
kubectl get endpoints nginx-service
tho new backend Pod ni chudachu.

Part 25 — Ingress

Ippudu manaki **Service + Load Balancer** already unnayi.
Ippudu **Ingress enduku kavalo** practical ga chuddam.
Step 1 — Problem enti?
Suppose future lo manaki 2 applications unnayi:

- nginx
- myapp

Rendu internet nunchi access cheyyali.
Prathi application ki separate LoadBalancer Service create chesthe:
nginx → Load Balancer 1
myapp → Load Balancer 2
Ila multiple Load Balancers avthai.
Enduku?
Unnecessary cost + management complexity.
Kabatti **Ingress** use chestham.

Step 2 — Ingress ante enti?
Ingress ante:
Internet nunchi vachina request ni URL/domain batti correct Service ki pampinche routing layer.
Example:
www.example.com/nginx
→ nginx-service
www.example.com/app
→ myapp-service
So Ingress ki:
“Ee URL request ee Service ki pampali?”
ani rules cheptham.

Step 3 — Ingress Controller kavali
Important:
Ingress YAML okkate saripodu.
Cluster lo **Ingress Controller** undali.
Ingress Controller actual ga incoming traffic ni receive chesi rules prakaram route chestundi.
AWS EKS lo common option:
AWS Load Balancer Controller

Step 4 — AWS Load Balancer Controller install cheyyali
Idi EKS cluster lo install chestham.
Enduku?
Kubernetes Ingress rules ni AWS Load Balancer rules tho connect cheyyadaniki.
Ikkada installation konchem lengthy: IAM permissions, service account, Helm etc. untayi.
So **ippudu actual installation commands loki vellakunda**, concept clear ga uncham.

Step 5 — Ingress YAML create cheyyali
File:
nginx-ingress.yaml
Example:

```
apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:
  name: nginx-ingress

spec:
  rules:
    - host: nginx.example.com
      http:
        paths:
```

```
    - path: /
      pathType: Prefix
      backend:
       service:
         name: nginx-service
         port:
          number: 80
```

Enduku?
Ingress ki rule chepthunnam:
nginx.example.com ki request vaste nginx-service ki pampu.

Step 6 — Ingress create cheyyali
kubectl apply -f nginx-ingress.yaml
Enduku?
Ingress rule ni Kubernetes cluster lo create cheyyadaniki.

Step 7 — Ingress create ayyinda check cheyyali
kubectl get ingress
Ingress address/hostname kanipistundi.

Step 8 — User request ela travel avtundi?
User:
nginx.example.com
open chestadu.
Request:
User
→ **AWS Load Balancer**
→ **Ingress Controller**
→ nginx.example.com rule check
→ **nginx-service**
→ **nginx Pod**
→ **nginx container**
→ nginx response.

Service vs Ingress simple ga
Service:
“Naa Pods ki traffic pampu.”
Ingress:
“Incoming request **ekkadiki** pampalo URL/domain batti decide cheyyi.”
So ippudu architecture:
User
→ **Load Balancer**
→ **Ingress**
→ **Service**
→ **Pods**
→ **Containers**
Part 25 complete.

1. _____
   ___

Part 26 — ConfigMap

Step 1 — ConfigMap create cheyyali
Em cheyyali?
nginx-config.yaml file create cheyyali.
Enduku?
Application configuration ni image lo hard-code cheyyakunda Kubernetes lo separate ga pettadaniki.
Example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  APP_ENV: "production"
```

Step 2 — Create
kubectl apply -f nginx-config.yaml
Result:
Kubernetes lo:
nginx-config
create avtundi.
Check:

kubectl get configmap

Part 27 — Secret
Step 1 — Secret create cheyyali
Enduku?
Password/API key/token lanti sensitive values ni Deployment YAML lo direct ga pettakunda separate ga manage cheyyadaniki.
Example:

```
kubectl create secret generic app-secret \
 --from-literal=DB_PASSWORD='mypassword'
```

Step 2 — Check

```
kubectl get secrets
```

Result:
app-secret Kubernetes lo create avtundi.
App container ki aa Secret value environment variable/file laga provide cheyyachu.

Part 28 — Storage: PV / PVC
Idi **nginx static website ki compulsory kaadu**.
Database/file storage kavalsina application anuko.
Step 1 — PVC create cheyyali
PVC = application ki storage request.
Example:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: app-storage
spec:
 accessModes:
  - ReadWriteOnce
 resources:
  requests:
   storage: 10Gi
```

Create:

```
kubectl apply -f pvc.yaml
```

Check:

```
kubectl get pvc
```

Result:
Kubernetes storage request create chestundi; EKS/AWS storage integration configured unte actual AWS storage provision avtundi.

Part 29 — HPA
Ippudu manaki Deployment already undi.
Suppose:
replicas: 3
Ippudu traffic perigite automatic ga Pods increase kavali.
Step 1 — Metrics available undali
HPA CPU/memory based scaling kosam cluster lo metrics system, commonly **Metrics Server**, available undali.
Step 2 — HPA create cheyyali

```
kubectl autoscale deployment nginx-deployment \
 --cpu-percent=70 \
 --min=3 \
 --max=10
```

Enduku?
CPU average 70% daggara/perigithe Kubernetes:
3 Pods → 4 → 5 → ...
increase cheyyachu.
Traffic/CPU taggithe:
5 → 4 → 3
decrease cheyyachu.
Check:

```
kubectl get hpa
```

Part 30 — Monitoring & Logging
Ikkada Kubernetes itself automatically full monitoring dashboard provide cheyyadu.
Production lo monitoring stack install chestam.
Common architecture:
Prometheus → metrics collect
Grafana → dashboards
Logs kosam:
CloudWatch / Fluent Bit lanti setup use cheyyachu.
Basic ga first Pod logs chudali ante:

```
kubectl logs <pod-name>
```

Pod status:

kubectl get pods
Detailed information:
kubectl describe pod <pod-name>
So monitoring ante:
Pod healthy unda? CPU entha? Memory entha? Errors unnaya?
ani observe cheyyadam.

Part 31 — CI/CD
Ippudu manual process:
Code
→ Docker build
→ ECR push
→ kubectl apply
Manam manually chestunnam.
CI/CD lo automated chestham.
Developer code push chestadu:
Git
↓
CI/CD pipeline
↓
Docker image build
↓
ECR push
↓
Kubernetes Deployment update
↓
New Pods rollout

Example pipeline tools
GitHub Actions / GitLab CI / Jenkins
lanti tools use cheyyachu.
Pipeline lo basic steps:

1. Code checkout
2. Docker image build
3. ECR login
4. Image push
5. Kubernetes deployment update
6. Rollout check

Deployment rollout check:
kubectl rollout status deployment/nginx-deployment

Part 32 — Deployment update
Suppose old image:
mynginx:1.0
New image:
mynginx:2.0
Deployment ni update chestam.

```
kubectl set image deployment/nginx-deployment \
  nginx=mynginx:2.0
```

Enduku?
Kubernetes ki:
“Ippudu new version image use cheyyi.”
ani cheppadaniki.
Deployment new ReplicaSet create chesi new Pods ni rollout chestundi.
Check:
kubectl rollout status deployment/nginx-deployment

Final complete flow
Ippudu manam conceptually complete production architecture ki vacham:
Developer
↓
Git
↓
CI/CD
↓
Dockerfile
↓
Docker Image
↓
ECR
↓

EKS
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Containers
↓
Service
↓
Ingress
↓
AWS Load Balancer
↓
User
Side lo:
ConfigMap → configuration
Secret → sensitive values
PVC → persistent storage
HPA → automatic Pod scaling
Monitoring → metrics/health
Logging → application/container logs
CI/CD → automatic build/deploy
Oka important point
Ivi anni prathi application ki compulsory kaavu.
Basic application ki:
ECR → EKS → Deployment → Pods → Service → Ingress
chaalu.
Production requirement batti:
ConfigMap / Secret / Storage / HPA / Monitoring / CI/CD
add chestham.